INFORMATION RETRIEVAL (CS60092)

Programming Assignment II

Vector Space Model and Ranked Retrieval

Sparse Vector-Space Retrieval on the Cranfield Collection

Team Name: MaamTeam

Roll No.	Name
23CS30035	Tanmai Raghava
23CS10013	Yesheeth Sree Narayana
23CS10055	Lokeswara Reddy
23CS30016	Manijith Dameruppala
23CS10011	Vivek Vardhan Varma

Contents

Executive Summary **3**

1 Problem Statement and Objectives **4**

1.1 Objectives 4

2 Collection and Data Preparation **4**

2.1 Collection Profile 4

2.2 Parser Behavior and Data Quirks 5

2.3 Preprocessing Chain 5

3 Implementation Details **5**

3.1 Terrier TF_IDF Baseline 5

3.2 SMART Weighting 6

3.3 Term Frequency 6

3.4 Inverse Document Frequency 6

3.5 Pivoted Unique-Term Normalization 7

3.6 Rocchio Pseudo-Relevance Feedback 7

3.7 Evaluation Implementation 7

4 Plan of Experiments **8**

4.1 Evaluation Metrics 8

4.2 Selection Principle 8

4.3 Reproducibility Note 8

5 Results – Preprocessing **9**

6 Results – SMART Weighting **10**

7 Results – Normalization and Rocchio Feedback **11**

7.1 Pivot-Slope Diagnostic 11

7.2 Rocchio Feedback Sweep 11

8 Results – Final System and Baseline Comparison **12**

9 Generalization, Efficiency and Engineering Analysis **13**

10 Discussion of Results **14**

10.1 Effect of Preprocessing 14

10.2 Effect of SMART Weighting 14
10.3 Feedback Behavior 15
10.4 Metric Trade-offs 15
10.5 Baseline Comparison 15
10.6 Limitations and Threats to Validity 15

11 Conclusion **16**

A Detailed Recorded Results **17**

A.1 Preprocessing Sweep 17
A.2 SMART Scheme Sweep 17

B Rocchio and Final Evaluation Data **18**

B.1 Rocchio Feedback Sweep 18
B.2 Final Baseline Comparison 18
B.3 Runtime Records 18

C Reproducibility and Implementation Walkthrough **18**

C.1 Core Data Structures 18
C.2 Execution Flow 19
C.3 Recommended Final Execution 19
C.4 Generated Artifacts 19

Summary

This report presents a reproducible sparse ranked-retrieval study on the Cranfield aerodynamics collection. Two retrieval pipelines were implemented and compared: a Terrier **TF_IDF** baseline and a custom, inspectable **SMART Vector Space Model (VSM)**. The custom pipeline supports configurable preprocessing, SMART term weighting, document normalization and Rocchio pseudo-relevance feedback. The assignment requires sparse vector-space models, comparative model and parameter experiments, and reporting of search and indexing time.

The recorded all-topic result shows that **SMART lnc.ltc + Rocchio** achieves **MAP = 0.3253**, compared with **0.2917** for Terrier TF_IDF. This is an absolute gain of **0.0336 MAP** and a relative gain of approximately **11.5%**. The selected SMART system also improves P@10, R@10, MRR, nDCG@10, nDCG@20 and R-Precision. The held-out 75-topic MAP is 0.3329, providing an additional generalization view.

The strongest non-feedback configuration is lnc.ltc with MAP 0.2998. Rocchio increases the best training MAP to 0.3216, an absolute gain of 0.0218 (about 7.3%). The best feedback setting retains the original query fully ($\alpha = 1.0$), uses eight positive feedback documents and 30 expansion terms, and uses no negative feedback.

A notable experimental diagnostic is that the recorded pivot-slope sweep is identical for all tested slopes. This is not evidence that pivot normalization is ineffective; rather, the selected lnc.ltc configuration uses cosine normalization, so the pivot slope is inactive. This is explicitly treated as an implementation/design finding and not over-interpreted.

Key Outcomes

Table 1. Summary of principal outcomes

Outcome	Recorded finding
Best all-topic MAP	**0.3253** – SMART lnc.ltc + Rocchio
Terrier baseline MAP	0.2917
Absolute MAP gain	**+0.0336**
Relative MAP gain	**+11.5%**
Best held-out MAP	0.3329 on 75 held-out topics
Best training MAP	0.3216 with Rocchio
Query-time comparison	≈10.3 ms/query SMART vs 35.23 ms/query Terrier

The assignment specifies that the report should cover group details, problem statement, implementation details, experimental plan, results and discussion of results. It also requires program files, result files and a PDF report.

1 Problem Statement and Objectives

The task is to build ranked-retrieval pipelines over the Cranfield text collection and evaluate them against human relevance judgments. The central research question is:

> *How much can careful sparse vector-space weighting, preprocessing and pseudo-relevance feedback improve ranked retrieval compared with a standard Terrier TF_IDF baseline?*

The project deliberately remains within the assignment boundary. No BM25, language-model retrieval, learning-to-rank, dense embeddings, neural retrieval or external relevance feedback is used. The two systems therefore provide a controlled comparison between a mature sparse search-engine implementation and a transparent custom sparse VSM.

1.1 Objectives

- Determine the effect of stemming, stopword removal, title weighting, field inclusion and title deduplication.
- Compare classical SMART weighting schemes such as lnc.ltc, ltc.ltc and lnu.ltu.
- Investigate document-length normalization and identify when pivoted normalization is actually active.
- Tune Rocchio pseudo-relevance feedback and quantify its contribution.
- Compare the selected SMART system with Terrier TF_IDF using multiple ranking metrics.
- Record indexing and query-time behavior.
- Maintain a deterministic and reproducible execution path.

2 Collection and Data Preparation

2.1 Collection Profile

Table 2. Cranfield collection used in the experiments

Resource	Count / description
Documents	1,400
Queries	225
Positive qrel records	1,612 in the project validation summary
Raw qrel records	1,837 including non-positive entries
Document structure	Cranfield .I, .T, .A, .B, .W records
Query structure	Cranfield .I and .W records

Documents contain an identifier, title, author, bibliography field and abstract/body field. Queries use raw identifiers that are not necessarily consecutive. The qrel file identifies queries positionally, so the parser maps qrel position 1 to the first query record, qrel position 2 to the second query record, and so on.

2.2 Parser Behavior and Data Quirks

The parser handles several collection-specific issues that can silently corrupt an experiment:

1. **Repeated field markers:** repeated fields such as .A or .W are accumulated instead of overwritten.
2. **Non-contiguous query identifiers:** raw query IDs are preserved and positional qrel identifiers are translated before scoring.
3. **Negative qrel entries:** negative relevance entries are skipped when constructing positive evaluation qrels.
4. **Duplicate identifiers:** duplicate document and query identifiers are rejected.
5. **Empty query records:** queries without usable text raise an error.

2.3 Preprocessing Chain

The custom SMART pipeline applies:

1. lowercase the input text;
2. extract alphanumeric tokens while retaining meaningful digits;
3. optionally remove stopwords;
4. optionally apply Porter stemming;
5. remove tokens shorter than two characters unless numeric;
6. construct weighted title, body, author and bibliography fields.

The title can be repeated multiple times through `title_weight`. This is an explicit field-boosting mechanism. The `dedupe_title` option removes a repeated title prefix from an abstract when present.

3 Implementation Details

The implementation is organized into separate data, retrieval, evaluation, I/O and orchestration components. All experiment settings are stored in `configs/experiments.yaml`, while `scripts/run_all.py` is responsible for parsing data, constructing the topic split, executing experiment stages, running Terrier, selecting the best SMART configuration, evaluating final runs and writing result/timing artifacts.

3.1 Terrier TF_IDF Baseline

The baseline uses PyTerrier's `IterDictIndexer` and Terrier's `TF_IDF` weighting model. The recorded baseline configuration is:

Table 3. Terrier baseline configuration

Parameter	Value
Terrier model	`TF_IDF`
Stopwords	Enabled
Stemming	Porter
Normalization parameter	$c = 1.0$
Default result depth	`top_k=1000`

The Terrier index is persisted at the configured index path. The `-force-index` option is used when collection preprocessing or Terrier settings change.

3.2 SMART Weighting

The custom model follows three-character SMART notation.

Table 4. SMART notation used in the implementation

Position	Meaning	Implemented choices
First	Term-frequency weighting	n = raw, l = logarithmic, b = binary
Second	Document-frequency weighting	n = none, t = IDF, p = probabilistic-style IDF
Third	Normalization	n = none, c = cosine, u = pivoted unique-term

Examples include lnc for logarithmic term frequency, IDF and cosine document normalization; ltc for logarithmic term frequency, IDF and cosine normalization; and lnu for logarithmic term frequency, no document-frequency weighting and pivoted unique-term normalization.

3.3 Term Frequency

For logarithmic term frequency:

$$\mathrm{tf}(t, d) = 1 + \log f(t, d), \tag{1}$$

where $f(t, d)$ is the number of occurrences of term t in document d. Raw frequency uses $f(t, d)$ directly, while binary frequency maps every present term to one.

3.4 Inverse Document Frequency

The smoothed IDF used by the custom implementation is:

$$\mathrm{idf}(t) = \log\left(\frac{N + 1}{df(t) + 1}\right) + 1, \tag{2}$$

where N is the number of documents and $df(t)$ is the number of documents containing term t.

3.5 Pivoted Unique-Term Normalization

Pivoted normalization compensates for the fact that longer documents naturally contain more terms and therefore more opportunities to match a query. The denominator is based on the average number of unique terms and the unique-term count of the current document:

$$\text{pivot}(d) = (1 - s)\bar{u} + s|u_d|, \tag{3}$$

where s is the pivot slope, $\bar{u}$ is the collection-average unique-term count and $|u_d|$ is the unique-term count of document d.

The recorded sweep uses:

$$s \in \{0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40\}.$$

3.6 Rocchio Pseudo-Relevance Feedback

Rocchio feedback expands the query using top-ranked documents from an initial retrieval pass:

$$q' = \alpha q + \frac{\beta}{|D_r|} \sum_{d \in D_r} d - \frac{\gamma}{|D_n|} \sum_{d \in D_n} d. \tag{4}$$

Here, q is the original query vector, D_r is the assumed relevant feedback set, D_n is the optional negative feedback set, and α, β, γ control the contributions.

The implementation protects original query terms, adds the highest-weight expansion terms up to `fb_terms`, and supports multiple feedback iterations. The recorded best configuration uses no negative feedback.

3.7 Evaluation Implementation

Both retrieval engines return a shared ranked-result structure containing query ID, document ID, rank and score. The evaluator uses the same qrels and metric definitions for both systems.

The reported metrics are:

- MAP;
- P@5 and P@10;
- R@10;
- MRR;
- nDCG@10 and nDCG@20;
- R-Precision.

Binary relevance uses qrel grades 1–3 as relevant. Graded relevance is used for nDCG.

4 Plan of Experiments

A deterministic split is used for staged model selection. Topics satisfying `index % 3 != 0` form the 150-topic training set, while topics satisfying `index % 3 == 0` form the 75-topic held-out set.

Table 5. Experimental stages

Stage	Purpose	Search space
1. Preprocessing	Find effective text preparation	10 combinations of stemming, stopwords, title weight, fields and deduplication
2. SMART selection	Compare sparse weighting schemes	8 SMART schemes
3. Pivot tuning	Explore normalization slope	$s = 0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.40$
4. Rocchio tuning	Tune pseudo-relevance feedback	8 feedback configurations
5. Combined search	Cross strong SMART models with feedback	Configured SMART $\times$ feedback grid
Baseline	Reference comparison	Terrier TF_IDF, $c = 1.0$

4.1 Evaluation Metrics

Table 6. Evaluation metrics and interpretation

Metric	What it measures
MAP	Mean average precision across topics; primary effectiveness summary
P@5 / P@10	Precision among the first 5 / 10 retrieved documents
R@10	Recall among the first 10 results
MRR	Average reciprocal rank of the first relevant result
nDCG@10 / nDCG@20	Graded ranking quality near the top of the result list
R-Precision	Precision at the number of known relevant documents

4.2 Selection Principle

The staged search is greedy rather than a full joint optimization. Within each stage, configurations are compared on training topics and the strongest configuration is carried forward. The held-out and all-topic evaluations provide checks against interpreting training performance in isolation.

4.3 Reproducibility Note

The supplied results document records that the stored stage tables were generated before a final Porter-stemming cleanup. Therefore, the numerical values in this report are presented as the **recorded experiment artifacts**. For strict final reproducibility, the complete runner should be executed once more after all code changes, and regenerated outputs should replace the recorded values if they differ.

5 Results – Preprocessing

The preprocessing sweep shows a clear hierarchy: configurations that retain title information and use stemming dominate lower-ranked alternatives. The best recorded MAP is **0.2998** for stemming + stopword removal + title ×2 + author + bibliography.

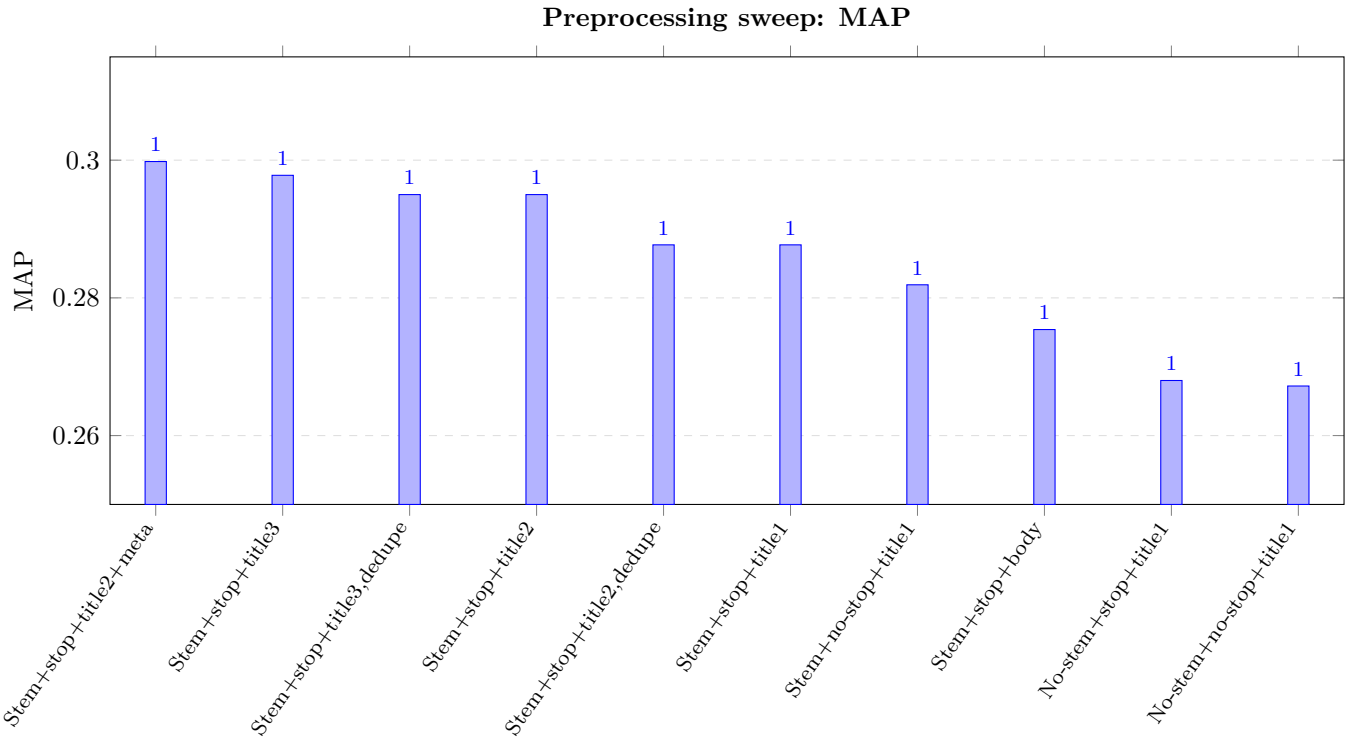


Figure 1. MAP across the ten recorded preprocessing configurations.

Interpretation. Adding useful metadata to the title/body representation provides the strongest recorded configuration. Removing stemming drops MAP from 0.2998 to 0.2680 when comparing the title ×1 variants, while removing stopwords also hurts performance. Body-only retrieval reaches 0.2754, demonstrating that titles carry useful topical signal. Title deduplication reduces MAP in the recorded runs, indicating that repeated title text acts as an effective implicit boost in this collection.

A useful nuance is that the top configurations are close: title ×2 + metadata (0.2998), title ×3 (0.2978), and title ×3 with deduplication (0.2950). The result therefore does not justify arbitrarily increasing title weight; it supports a moderate title boost plus informative metadata.

Table 7. Selected preprocessing comparisons

Configuration	MAP	Change vs. best
Best preprocessing	**0.2998**	–
Title ×3	0.2978	-0.0020
Title ×2, dedupe	0.2877	-0.0121
Body only	0.2754	-0.0244
No stem + stop + title ×1	0.2680	-0.0318

6 Results – SMART Weighting

The SMART sweep isolates the effect of term-frequency, IDF and normalization choices after preprocessing is fixed. The strongest MAP is 0.2998 for **lnc.ltc/Lnc.ltc**, closely followed by ltc.ltc and ltc.ltn at 0.2987. The binary document term-frequency scheme bnc.ltc is substantially weaker at 0.2361.

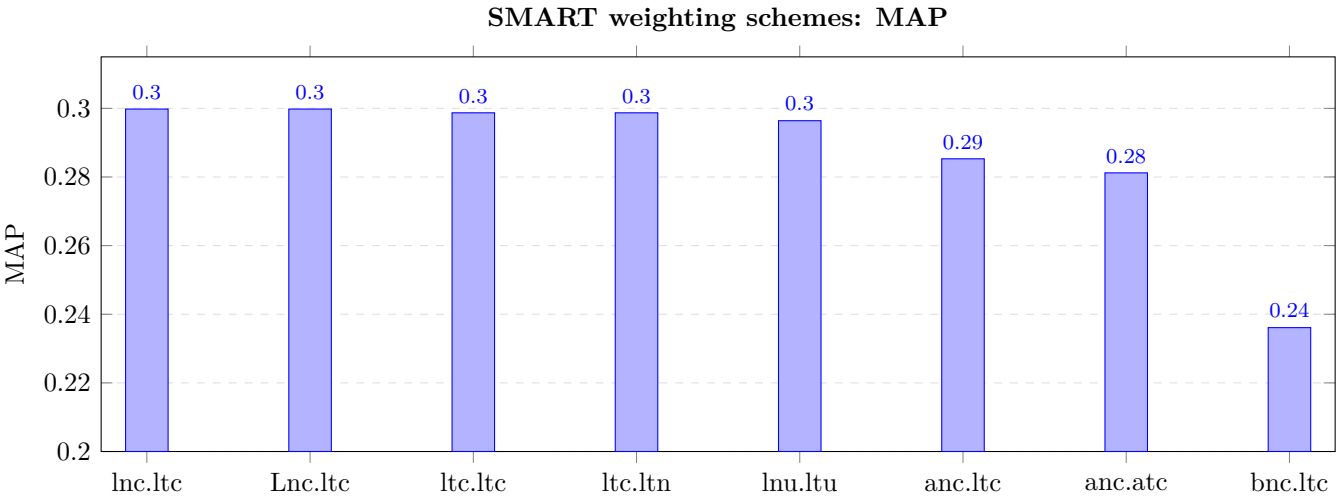


Figure 2. MAP comparison across recorded SMART schemes.

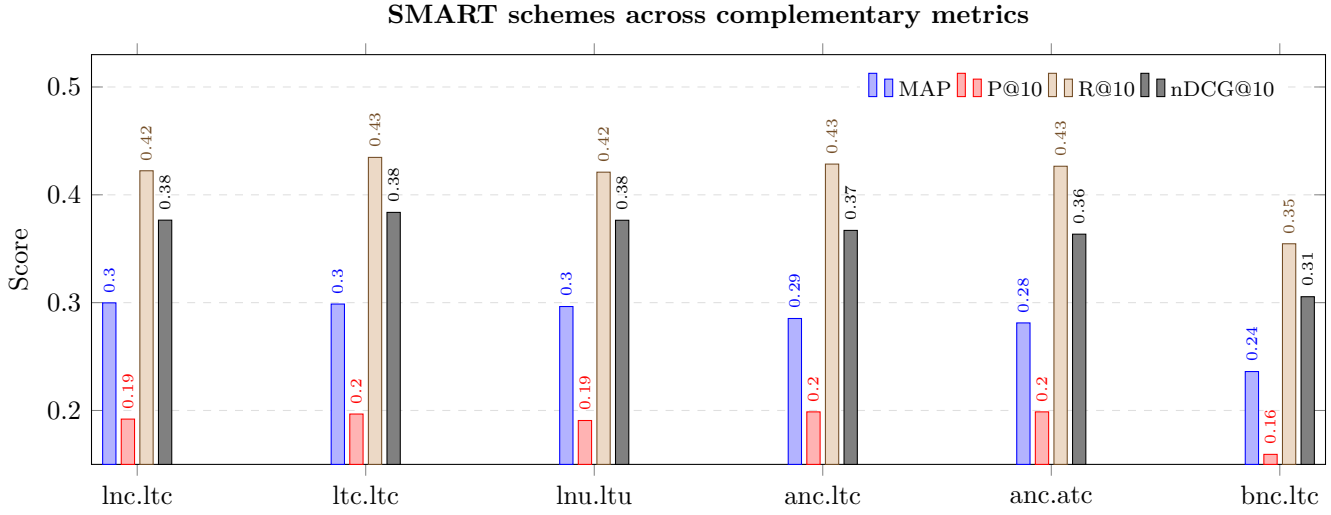


Figure 3. Complementary metrics show that the MAP winner is not the winner on every early-ranking measure.

Interpretation. Logarithmic term-frequency weighting preserves useful frequency information without allowing repeated terms to dominate linearly. IDF remains important for suppressing terms that occur in many documents. Binary weighting discards frequency information and shows the largest effectiveness loss. The lnu.ltu configuration obtains the highest P@5 and ties for the highest MRR, illustrating why multiple metrics are necessary when choosing a retrieval model.

7 Results – Normalization and Rocchio Feedback

7.1 Pivot-Slope Diagnostic

All seven recorded slopes (0.05 through 0.40) produce exactly the same lnc.ltc scores and metrics. This is expected because lnc.ltc uses **cosine normalization**, not **u = pivoted unique-term normalization**. Consequently, the slope parameter is not consulted by the selected normalization branch.

Table 8. Recorded pivot-slope sweep

Slope	MAP	P@10	R@10	nDCG@10
0.05	0.2998	0.1920	0.4223	0.3765
0.10	0.2998	0.1920	0.4223	0.3765
0.15	0.2998	0.1920	0.4223	0.3765
0.20	0.2998	0.1920	0.4223	0.3765
0.25	0.2998	0.1920	0.4223	0.3765
0.30	0.2998	0.1920	0.4223	0.3765
0.40	0.2998	0.1920	0.4223	0.3765

This is a useful implementation finding rather than a negative experimental result. A scientifically active pivot experiment should instead use a scheme containing `u`, such as lnu.ltu.

7.2 Rocchio Feedback Sweep

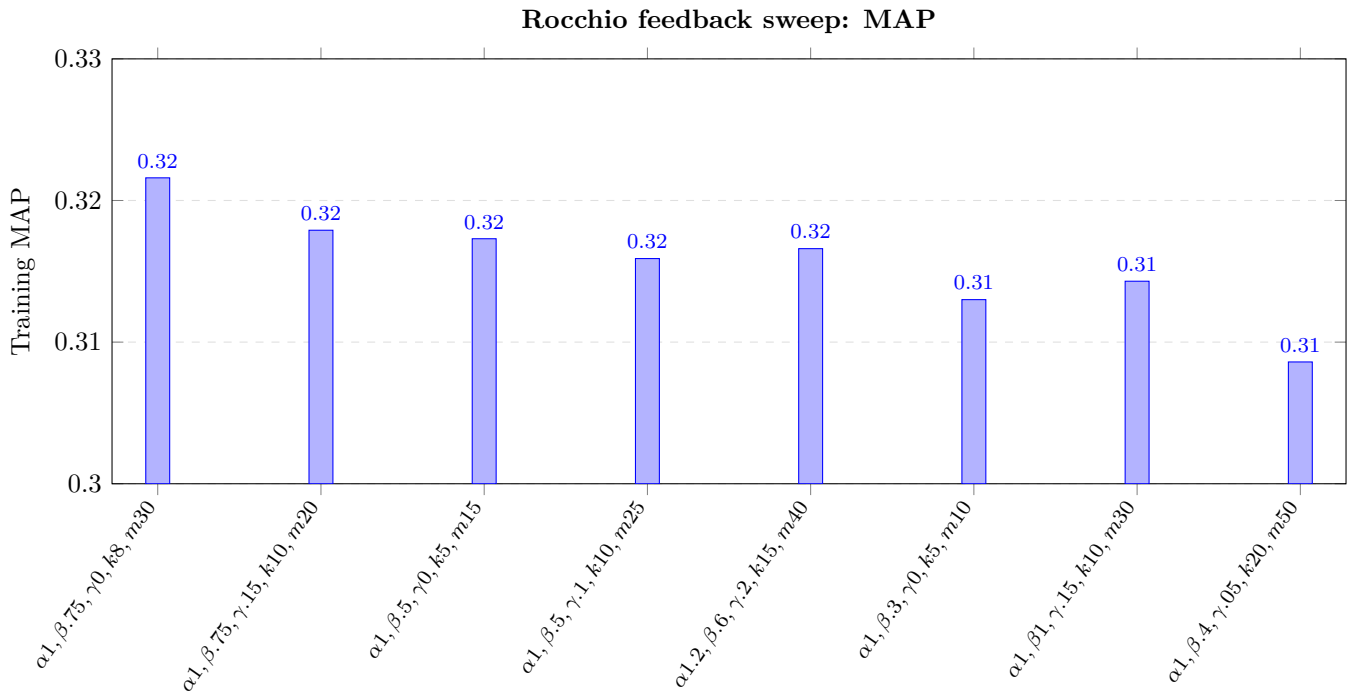


Figure 4. Training MAP for the eight recorded Rocchio configurations.

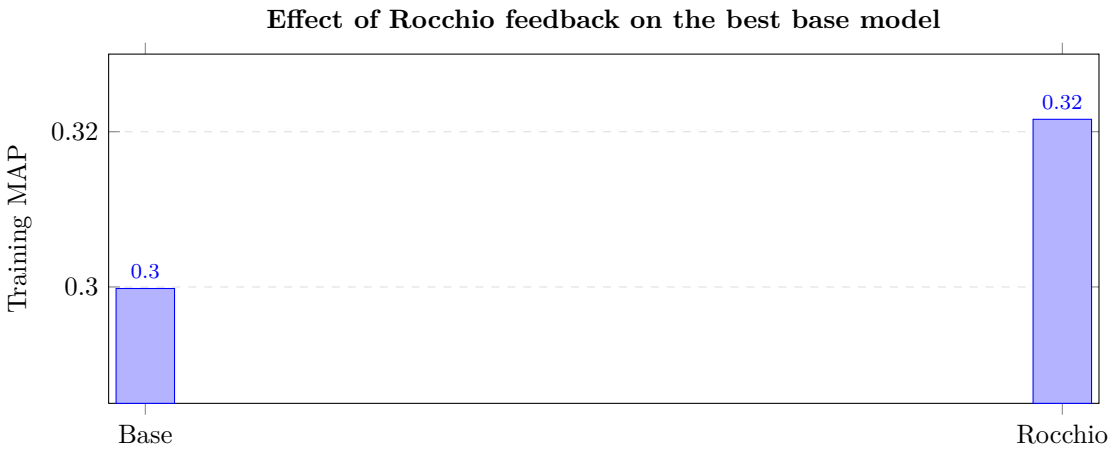


Figure 5. Direct gain from adding the best recorded Rocchio configuration.

The best feedback setting reaches MAP 0.3216, compared with 0.2998 for the best base model. This is a **0.0218 absolute gain**, or about **7.3% relative improvement** on the training set. The best setting uses $\alpha = 1.0$, $\beta = 0.75$, $\gamma = 0$, $k = 8$ feedback documents and $m = 30$ expansion terms.

The full preservation of the original query limits query drift, while the absence of negative feedback suggests that low-ranked documents are not reliable enough to act as strong anti-evidence in this collection.

8 Results – Final System and Baseline Comparison

The combined stage crosses the strongest SMART schemes with the configured feedback grid. The best recorded training configuration is **lnc.ltc + Rocchio(1.0, 0.75, 0.0, 8, 30)**, with MAP 0.3216.

Table 9. Best configurations from the combined stage

System	MAP	P@5	P@10	R@10	MRR	nDCG@10
lnc.ltc + Rocchio	**0.3216**	0.2707	0.2120	0.4582	**0.5064**	**0.4006**
Lnc.ltc + same feedback	**0.3216**	0.2707	0.2120	0.4582	**0.5064**	**0.4006**
ltc.ltc + same feedback	0.3214	**0.2800**	0.2113	**0.4611**	0.4967	0.3996

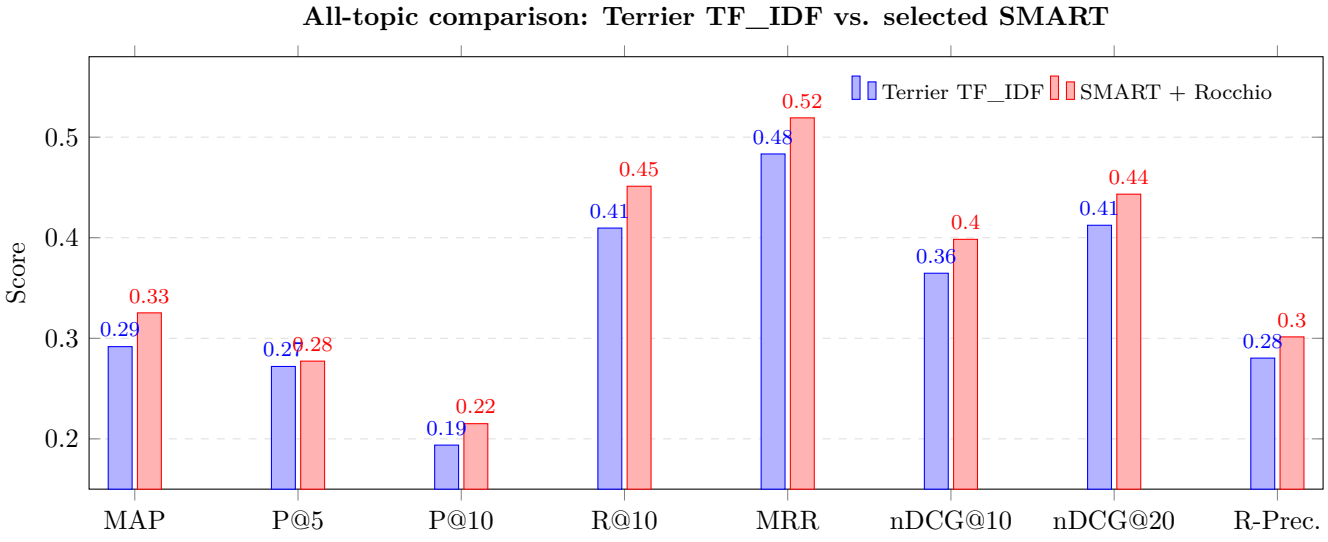


Figure 6. All-topic effectiveness comparison.

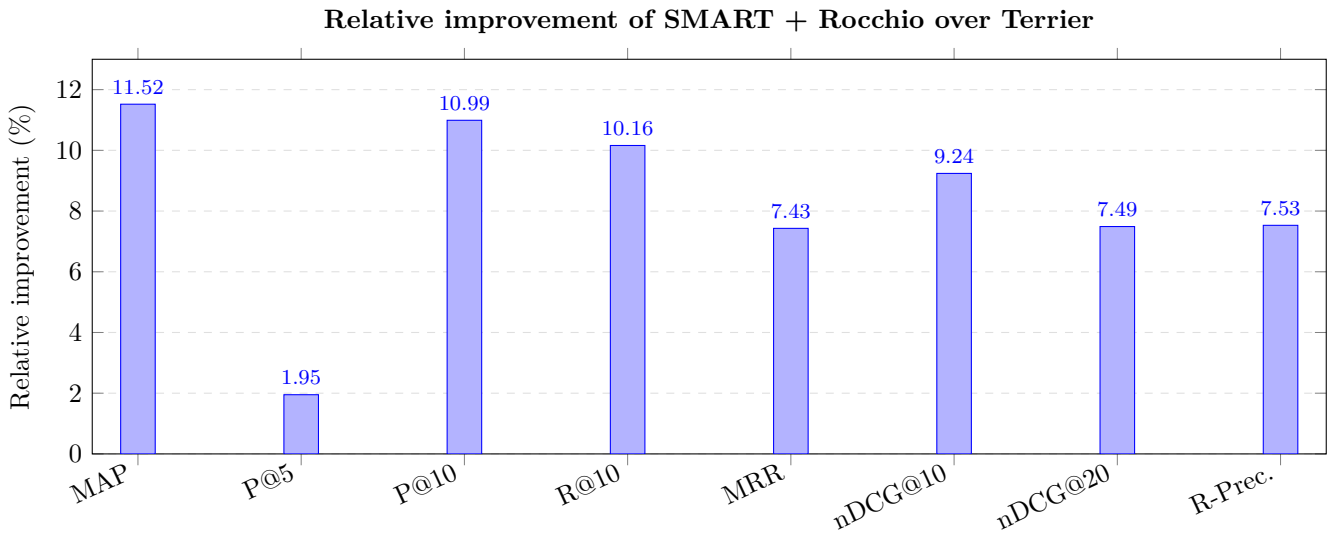


Figure 7. Relative improvement across all reported metrics.

Key insight. The selected SMART system improves every reported all-topic metric in the supplied results. The largest relative gain is in **MAP** (about 11.5%), followed closely by P@10 (about 11.0%) and R@10 (about 10.2%). The improvement is therefore not confined to one evaluation measure; the feedback-enhanced system both retrieves more relevant material early and improves overall ranking quality.

9 Generalization, Efficiency and Engineering Analysis

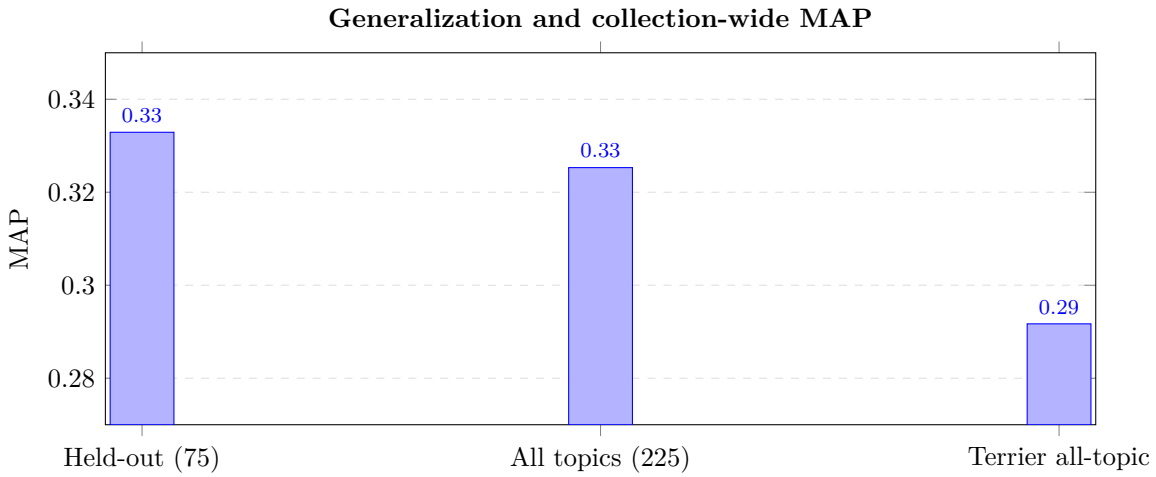


Figure 8. MAP on the held-out set, all topics, and the Terrier all-topic baseline.

The selected SMART system obtains **0.3329 MAP** on the held-out 75 topics and **0.3253 MAP** across all 225 topics. The held-out score is higher than the collection-wide score by 0.0076 MAP. Because the split is deterministic and finite, this difference should not be interpreted as proof of stronger generalization; it is best reported as an additional validation view.

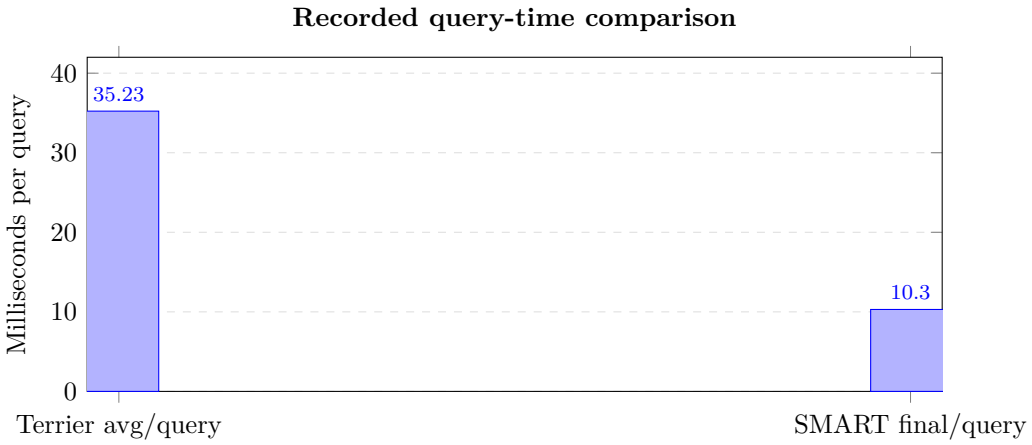


Figure 9. Recorded query-time comparison.

The recorded Terrier average query time is 35.23 ms/query, while the final SMART configuration is approximately 10.3 ms/query. That corresponds to roughly **3.4× lower query latency** for SMART in the recorded comparison.

Table 10. Runtime records

Runtime component	Recorded value
Terrier indexing	27.73 s
Terrier query time for 225 topics	7.93 s
Terrier average query	35.23 ms/query
SMART final configuration	≈10.3 ms/query
SMART stage range	≈2.0–12.1 ms/query

Terrier nevertheless provides a mature persistent indexing/search framework, while the custom SMART implementation benefits from direct in-memory postings and cached document vectors, avoiding JVM calls during individual scoring operations.

10 Discussion of Results

10.1 Effect of Preprocessing

Preprocessing has a measurable effect before model selection begins. Stemming improves term matching across morphological variants, while stopword removal reduces common-word noise. Titles provide strong topical cues, and the recorded best configuration benefits further from author and bibliography fields.

The result is not simply “more text is better”: body-only retrieval is weaker, and title duplication helps only within a moderate range.

10.2 Effect of SMART Weighting

The strongest schemes use logarithmic term frequency and IDF. This supports the intuition that repeated occurrences of a query term are useful evidence, but their contribution should saturate rather than grow linearly.

Binary weighting performs markedly worse, indicating that frequency information is valuable for discriminating focused documents from documents that merely mention a term.

10.3 Feedback Behavior

Rocchio feedback improves the best training MAP from approximately 0.2998 to 0.3216. The 0.0218 MAP gain is larger than the differences among the strongest non-feedback SMART schemes. This suggests that, for this collection, query expansion is more consequential than fine distinctions among already-competitive term-weighting variants.

The best feedback configuration uses eight positive feedback documents and thirty expansion terms. The original query receives full weight through $\alpha = 1$, which limits query drift. Negative feedback is disabled in the best result.

10.4 Metric Trade-offs

MAP is the primary selection metric in the staged experiment, but ltc.ltc and lnu.ltu achieve stronger P@10, R@10 or MRR in individual comparisons. A model can therefore be preferable for a specific application even when it is not the MAP winner.

Reporting multiple metrics is essential because users often care disproportionately about the first few retrieved documents.

10.5 Baseline Comparison

The all-topic selected SMART system outperforms Terrier TF_IDF on MAP, recall at 10, MRR and both nDCG cutoffs, as well as the other reported metrics. This is meaningful evidence that transparent sparse weighting plus pseudo-relevance feedback can improve the baseline on Cranfield.

The comparison should remain collection-specific. Terrier and the custom system also have different internal tokenization and indexing implementations, despite aligned high-level preprocessing choices.

10.6 Limitations and Threats to Validity

- The staged search is greedy rather than a full joint optimization.
- The stored tables predate the final Porter-stemming cleanup and should be regenerated for strict reproducibility.
- The pivot sweep is inactive because it was run with lnc.ltc rather than a u-normalized scheme.
- Terrier and SMART use different internal indexing/tokenization implementations.
- The held-out split is deterministic rather than randomly stratified.
- Runtime depends on JVM startup, hardware, memory pressure and file-system behavior.

11 Conclusion

This project delivers a unified sparse ranked-retrieval framework for the Cranfield collection. It combines a standard Terrier TF_IDF baseline with an inspectable custom SMART VSM implementation, a configurable preprocessing pipeline, multiple SMART weighting choices, Rocchio pseudo-relevance feedback, shared evaluation logic and reproducible experiment orchestration.

The central empirical finding is clear: the selected **SMART lnc.ltc + Rocchio** system records **MAP 0.3253** on all 225 topics, improving over the Terrier baseline's **0.2917**. The absolute MAP gain is **0.0336**, approximately **11.5%** relative. Improvements are also observed in early precision, recall, reciprocal rank and graded ranking quality.

Engineering-wise, the project demonstrates the value of separating parsing, retrieval, evaluation and experiment orchestration. The same qrel alignment and metrics can be reused across engines, reducing accidental differences. The recorded query-time results also indicate that the in-memory SMART implementation can be substantially lighter during scoring.

A Detailed Recorded Results

The appendix keeps the main narrative readable while preserving the complete numerical data used for the principal plots.

A.1 Preprocessing Sweep

Table 11. Complete preprocessing sweep

Configuration	MAP	P@5	P@10	R@10	MRR	nDCG@10
Stem + stop + title ×2 + author + bibliography	.2998	.2693	.1920	.4223	.4954	.3765
Stem + stop + title ×3	.2978	.2693	.1920	.4281	.4969	.3782
Stem + stop + title ×3, dedupe	.2950	.2707	.1907	.4190	.4929	.3720
Stem + stop + title ×2	.2950	.2707	.1907	.4190	.4929	.3720
Stem + stop + title ×2, dedupe	.2877	.2680	.1900	.4154	.4754	.3647
Stem + stop + title ×1	.2877	.2680	.1900	.4154	.4754	.3647
Stem + no stop + title ×1	.2819	.2587	.1827	.4098	.4629	.3582
Stem + stop + body only	.2754	.2453	.1807	.3930	.4654	.3476
No stem + stop + title ×1	.2680	.2453	.1853	.4002	.4644	.3504
No stem + no stop + title ×1	.2672	.2507	.1760	.3837	.4608	.3445

A.2 SMART Scheme Sweep

Table 12. Complete SMART scheme sweep

Scheme	MAP	P@5	P@10	R@10	MRR	nDCG@10
lnc.ltc	.2998	.2693	.1920	.4223	.4954	.3765
Lnc.ltc	.2998	.2693	.1920	.4223	.4954	.3765
ltc.ltc	.2987	.2707	.1967	.4347	.5054	.3837
ltc.ltn	.2987	.2707	.1967	.4347	.5054	.3837
lnu.ltu	.2964	.2787	.1907	.4210	.5054	.3764
anc.ltc	.2853	.2627	.1987	.4285	.4646	.3670
anc.atc	.2812	.2587	.1987	.4265	.4588	.3635
bnc.ltc	.2361	.2000	.1593	.3546	.4078	.3055

B Rocchio and Final Evaluation Data

B.1 Rocchio Feedback Sweep

Table 13. Complete Rocchio sweep

Configuration	MAP	P@5	P@10	R@10	MRR	nDCG@10
$(1.0, .75, 0, 8, 30)$	.3216	.2707	.2120	.4582	.5064	.4006
$(1.0, .75, .15, 10, 20)$	.3179	.2680	.2073	.4411	.5083	.3930
$(1.0, .5, 0, 5, 15)$	.3173	.2733	.2160	.4689	.4935	.3999
$(1.0, .5, .1, 10, 25)$	.3159	.2733	.2047	.4398	.5062	.3915
$(1.2, .6, .2, 15, 40)$	.3166	.2787	.2080	.4492	.4992	.3919
$(1.0, .3, 0, 5, 10)$	.3130	.2747	.2093	.4542	.5034	.3938
$(1.0, 1, .15, 10, 30)$	.3143	.2707	.2100	.4484	.4992	.3942
$(1.0, .4, .05, 20, 50)$	.3086	.2787	.2033	.4391	.4905	.3840

B.2 Final Baseline Comparison

Table 14. Final Terrier versus SMART results

System	Set	MAP	P@5	P@10	R@10	MRR	nDCG@10	nDCG@20	R-Prec.
Terrier TF_IDF	All 225	.2917	.2720	.1938	.4096	.4833	.3647	.4124	.2803
SMART + Rocchio	Held-out 75	.3329	.2907	.2213	.4370	.5448	.3941	.4291	.3265
SMART + Rocchio	All 225	**.3253**	**.2773**	**.2151**	**.4512**	**.5192**	**.3984**	**.4433**	**.3014**

B.3 Runtime Records

Table 15. Recorded runtime values

Component	Value
Terrier indexing	27.73 s
Terrier query time for 225 topics	7.93 s
Terrier average query	35.23 ms/query
SMART final configuration	≈10.3 ms/query
SMART stage query range	≈2.0–12.1 ms/query

C Reproducibility and Implementation Walkthrough

C.1 Core Data Structures

The shared data contract contains `Document`, `Query`, `Qrel` and `RankedResult` objects. The SMART index maintains a forward document-vector cache for Rocchio feedback and an inverted postings structure for query-time scoring.

C.2 Execution Flow

1. Load YAML configuration.
2. Parse and validate the Cranfield collection.
3. Create training and held-out topic lists.
4. Run preprocessing, SMART, pivot, Rocchio and combined stages.
5. Build and query the Terrier baseline.
6. Select the best SMART configuration by training MAP.
7. Evaluate on held-out and all topics.
8. Write CSV/JSON summaries, TREC runs and timing artifacts.

C.3 Recommended Final Execution

The recorded complete-run command is:

```
python scripts/run_all.py
  -config configs/experiments.yaml
  -data-dir data
  -results-dir results
  -benchmarks-dir benchmarks
  -index-path indexes/terrier_tfidf_baseline
  -force-index
```

The `-force-index` option should be used when collection preprocessing or Terrier settings have changed, so that an old persisted index is not accidentally reused.

C.4 Generated Artifacts

Table 16. Expected generated artifacts

Artifact	Purpose
`MaamTeam_stage1_preprocessing.csv`	Preprocessing sweep
`MaamTeam_stage2_models.csv`	SMART scheme comparison
`MaamTeam_stage3_parameters.csv`	Pivot-slope sweep
`MaamTeam_stage4_rocchio.csv`	Feedback sweep
`MaamTeam_stage5_best_combination.csv`	Combined SMART search
`MaamTeam_terrier_tfidf.txt`	Terrier TREC run
`MaamTeam_results.txt`	Selected SMART TREC run
`MaamTeam_best_config.json`	Selected configuration and final metrics
`experiment_summary.csv/.json`	Unified experiment records
`timing.csv/.json`	Timing records